

# Logging Claude Code Conversations to a Database

Complete Setup Guide

macOS & Ubuntu • PostgreSQL & MySQL

*Works with both Claude Code CLI and the VS Code extension*

Tested & verified — captures user prompts, assistant replies,  
tool calls, and errors with race-condition handling.

# Overview

This guide shows you how to log every Claude Code conversation — every prompt you send, every response, every tool call, every error — into a database. The setup is global: configure it once on your machine and every project is covered automatically.

## How it works

Claude Code provides hooks: shell commands that fire on specific events such as a prompt being submitted, a response finishing, or a tool being used. We register a small Python script that captures each event and writes a row to the database.

Hooks are read from `~/.claude/settings.json` on your machine. Both the Claude Code CLI and the VS Code extension use the same engine and the same settings file, so a single global setup covers both interfaces.

## Events captured

| Event            | What gets logged                                                   |
|------------------|--------------------------------------------------------------------|
| UserPromptSubmit | Every prompt you send (role = user)                                |
| Stop             | Claude's final assistant response (role = assistant, agent = main) |
| SubagentStop     | Subagent responses with the agent's name                           |
| PreToolUse       | Tool name and input arguments before execution                     |
| PostToolUse      | Tool output, plus error flag and message if the tool failed        |
| Notification     | System notifications (permission prompts, idle alerts)             |
| SessionStart     | New session — populates the sessions table                         |

## What you'll need

- A working Claude Code installation (CLI or VS Code extension)
- Python 3 with pip
- PostgreSQL or MySQL installed and running
- Terminal access

## Choose your path

Pick the section that matches your operating system and database. Each section is self-contained — follow it top to bottom.

- Section A — macOS + PostgreSQL
- Section B — macOS + MySQL
- Section C — Ubuntu + PostgreSQL
- Section D — Ubuntu + MySQL

After completing your section, jump to “Common steps” to install the logger script and register the hooks. Those steps are identical across all four combinations.

## Section A — macOS + PostgreSQL

### A.1 Install PostgreSQL

```
brew install postgresql@16
brew services start postgresql@16
```

### A.2 Create the database and user

```
psql postgres
```

Then inside the psql prompt:

```
CREATE USER claudie WITH PASSWORD 'your_strong_password';
CREATE DATABASE claudie_logs OWNER claudie;
\q
```

**Password tip:** Avoid characters like # ; = & in the password — they confuse various config parsers and will save you debugging time later. Stick to letters, digits, underscores, and hyphens.

### A.3 Create the tables

```
psql -U claudie -d claudie_logs -h localhost
```

Paste the schema below, then type \q to exit:

```
CREATE TABLE IF NOT EXISTS cc_sessions (
  session_id      TEXT PRIMARY KEY,
  started_at      TIMESTAMPTZ DEFAULT NOW(),
  last_seen_at    TIMESTAMPTZ DEFAULT NOW(),
  cwd             TEXT,
  project_dir     TEXT
);

CREATE TABLE IF NOT EXISTS cc_events (
  id              BIGSERIAL PRIMARY KEY,
  session_id      TEXT NOT NULL,
  timestamp       TIMESTAMPTZ DEFAULT NOW(),
  event_type      TEXT NOT NULL,
  agent          TEXT,
  role           TEXT,
  content         TEXT,
  tool_name       TEXT,
  tool_input      JSONB,
  tool_output     JSONB,
  is_error        BOOLEAN DEFAULT FALSE,
  error_message   TEXT,
  raw_payload     JSONB,
  transcript_path TEXT
);
```

```
CREATE INDEX IF NOT EXISTS idx_cc_events_session ON cc_events(session_id);
CREATE INDEX IF NOT EXISTS idx_cc_events_time     ON cc_events(timestamp);
CREATE INDEX IF NOT EXISTS idx_cc_events_type     ON cc_events(event_type);
CREATE INDEX IF NOT EXISTS idx_cc_events_errors   ON cc_events(is_error) WHERE is_error
= TRUE;
```

## A.4 Install the Python driver

```
pip3 install psycopg2-binary
```

Now jump to the “Common steps” section. You'll use the PostgreSQL variant of the logger script.

## Section B — macOS + MySQL

### B.1 Install MySQL

```
brew install mysql
brew services start mysql
```

Run the secure installation to set a root password:

```
mysql_secure_installation
```

### B.2 Create the database and user

```
mysql -u root -p
```

Inside the MySQL prompt:

```
CREATE DATABASE claudе_logs CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'claudе'@'localhost' IDENTIFIED BY 'your_strong_password';
GRANT ALL PRIVILEGES ON claudе_logs.* TO 'claudе'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

**Password tip:** Avoid # ; = & in the password — these can break various config parsers. Stick to letters, digits, underscores, and hyphens. Putting the password directly in the Python script (which we'll do later) sidesteps most parsing issues, but it's still cleanest to keep the password simple.

### B.3 Create the tables

```
mysql -u claudе -p claudе_logs
```

Paste the schema below, then type EXIT:

```
CREATE TABLE IF NOT EXISTS cc_sessions (
  session_id    VARCHAR(255) PRIMARY KEY,
  started_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_seen_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  cwd           TEXT,
  project_dir   TEXT
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS cc_events (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  session_id    VARCHAR(255) NOT NULL,
  timestamp     TIMESTAMP(3) DEFAULT CURRENT_TIMESTAMP(3),
  event_type    VARCHAR(64) NOT NULL,
  agent         VARCHAR(128),
  role          VARCHAR(32),
  content       LONGTEXT,
  tool_name     VARCHAR(128),
  tool_input    JSON,
  tool_output   JSON,
  is_error      BOOLEAN DEFAULT FALSE,
```

```
error_message TEXT,  
raw_payload JSON,  
transcript_path TEXT,  
INDEX idx_session (session_id),  
INDEX idx_time (timestamp),  
INDEX idx_type (event_type),  
INDEX idx_errors (is_error)  
) ENGINE=InnoDB;
```

## B.4 Install the Python driver

```
pip3 install mysql-connector-python
```

Now jump to the “Common steps” section. You'll use the MySQL variant of the logger script.

## Section C — Ubuntu + PostgreSQL

### C.1 Install PostgreSQL

```
sudo apt update
sudo apt install -y postgresql postgresql-contrib python3-pip
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

### C.2 Create the database and user

```
sudo -u postgres psql
```

Inside the psql prompt:

```
CREATE USER claudie WITH PASSWORD 'your_strong_password';
CREATE DATABASE claudie_logs OWNER claudie;
\q
```

**Password tip:** Avoid # ; = & in the password to keep things simple later. Use letters, digits, underscores, hyphens.

### C.3 Create the tables

```
psql -U claudie -d claudie_logs -h localhost
```

**Note:** If you get a peer authentication error, edit /etc/postgresql/\*/main/pg\_hba.conf and change the local line for claudie\_logs to use md5 or scram-sha-256, then restart with sudo systemctl restart postgresql.

Paste the schema:

```
CREATE TABLE IF NOT EXISTS cc_sessions (
    session_id    TEXT PRIMARY KEY,
    started_at    TIMESTAMPTZ DEFAULT NOW(),
    last_seen_at  TIMESTAMPTZ DEFAULT NOW(),
    cwd           TEXT,
    project_dir   TEXT
);

CREATE TABLE IF NOT EXISTS cc_events (
    id            BIGSERIAL PRIMARY KEY,
    session_id    TEXT NOT NULL,
    timestamp     TIMESTAMPTZ DEFAULT NOW(),
    event_type    TEXT NOT NULL,
    agent         TEXT,
    role          TEXT,
    content       TEXT,
    tool_name     TEXT,
    tool_input    JSONB,
    tool_output   JSONB,
    is_error      BOOLEAN DEFAULT FALSE,
```

```
error_message    TEXT,  
raw_payload      JSONB,  
transcript_path  TEXT  
);  
  
CREATE INDEX IF NOT EXISTS idx_cc_events_session ON cc_events(session_id);  
CREATE INDEX IF NOT EXISTS idx_cc_events_time    ON cc_events(timestamp);  
CREATE INDEX IF NOT EXISTS idx_cc_events_type    ON cc_events(event_type);  
CREATE INDEX IF NOT EXISTS idx_cc_events_errors  ON cc_events(is_error) WHERE is_error  
= TRUE;
```

## C.4 Install the Python driver

```
pip3 install psycopg2-binary
```

Now jump to the “Common steps” section. You’ll use the PostgreSQL variant of the logger script.

## Section D — Ubuntu + MySQL

### D.1 Install MySQL

```
sudo apt update
sudo apt install -y mysql-server python3-pip
sudo systemctl start mysql
sudo systemctl enable mysql
sudo mysql_secure_installation
```

### D.2 Create the database and user

```
sudo mysql
```

Inside the MySQL prompt:

```
CREATE DATABASE claudelogs CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'claudelogs'@'localhost' IDENTIFIED BY 'your_strong_password';
GRANT ALL PRIVILEGES ON claudelogs.* TO 'claudelogs'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

**Password tip:** Avoid # ; = & in the password. Stick to letters, digits, underscores, hyphens.

### D.3 Create the tables

```
mysql -u claudelogs -p claudelogs
```

Paste the schema:

```
CREATE TABLE IF NOT EXISTS cc_sessions (
  session_id    VARCHAR(255) PRIMARY KEY,
  started_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_seen_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  cwd            TEXT,
  project_dir   TEXT
) ENGINE=InnoDB;

CREATE TABLE IF NOT EXISTS cc_events (
  id            BIGINT AUTO_INCREMENT PRIMARY KEY,
  session_id    VARCHAR(255) NOT NULL,
  timestamp     TIMESTAMP(3) DEFAULT CURRENT_TIMESTAMP(3),
  event_type    VARCHAR(64) NOT NULL,
  agent         VARCHAR(128),
  role          VARCHAR(32),
  content       LONGTEXT,
  tool_name     VARCHAR(128),
  tool_input    JSON,
  tool_output   JSON,
  is_error      BOOLEAN DEFAULT FALSE,
  error_message TEXT,
  raw_payload   JSON,
```

```
transcript_path TEXT,  
INDEX idx_session (session_id),  
INDEX idx_time (timestamp),  
INDEX idx_type (event_type),  
INDEX idx_errors (is_error)  
) ENGINE=InnoDB;
```

## D.4 Install the Python driver

```
pip3 install mysql-connector-python
```

Now jump to the “Common steps” section. You'll use the MySQL variant of the logger script.

## Common steps (all systems)

These steps are identical regardless of OS or database. Just pick the variant of the logger script that matches your database when you reach Step 2.

### Step 1 — Create the Claude config directory

```
mkdir -p ~/.claude
```

### Step 2 — Create the logger script

Pick ONE of the two scripts below — the PostgreSQL version or the MySQL version — and save it as `~/.claude/log-to-db.py`.

**About the password:** We're putting the database password directly inside the Python script. It's the simplest, most reliable approach because it sidesteps option-file parsing differences between MySQL clients and the Python connector. The script is locked down to your user account only via `chmod 700`, and you'll see the security note at the end of this guide. Just don't commit this file to git.

**Why the script has retry logic:** When Claude Code finishes a response, the Stop hook fires the moment streaming completes — but the assistant's reply is written to the transcript file with a tiny delay. Without retry handling, the script reads the file too early and saves an empty assistant turn. The script retries up to 5 times with 300ms gaps, which solves this race condition cleanly.

### Option A — PostgreSQL logger script

Save this as `~/.claude/log-to-db.py`. Replace `YOUR_PASSWORD_HERE` with your actual password.

```
#!/usr/bin/env python3
"""Claude Code -> MySQL logger with debug + retry on transcript read."""
import sys, os, json, time, traceback, datetime

DB_CONFIG = {
    "user":      "claude",
    "password":  "YOUR_PASSWORD_HERE",  # <-- put your real password here
    "host":      "localhost",
    "port":      5432,
    "dbname":    "claude_logs",
}

DEBUG_LOG = os.path.expanduser("~/claude/logger-debug.log")
ERR_LOG   = os.path.expanduser("~/claude/logger-errors.log")

def dbg(msg):
    try:
        with open(DEBUG_LOG, "a") as f:
            f.write(f"[{datetime.datetime.utcnow().isoformat()}] {msg}\n")
```

```

except Exception:
    pass

def log_error(msg):
    try:
        with open(ERR_LOG, "a") as f:
            f.write(f"[{datetime.datetime.utcnow().isoformat()}] {msg}\n")
    except Exception:
        pass

dbg(f"=== script started, pid={os.getpid()}, python={sys.executable} ===")

try:
    import psycpg2
    from psycpg2.extras import Json
    dbg("psycpg2 imported OK")
except Exception as e:
    dbg(f"FAILED to import psycpg2: {e}")
    log_error(f"import failed: {e}\n{traceback.format_exc()}")
    sys.exit(0)

def _extract_text_from_content(content):
    if content is None:
        return None
    if isinstance(content, str):
        return content.strip() or None
    if isinstance(content, list):
        parts = []
        for c in content:
            if not isinstance(c, dict):
                continue
            ctype = c.get("type")
            if ctype == "text" and c.get("text"):
                parts.append(c["text"])
            elif ctype in ("thinking", "tool_use", "tool_result"):
                continue
            elif "text" in c and isinstance(c["text"], str):
                parts.append(c["text"])
        joined = "\n".join(p for p in parts if p).strip()
        return joined or None
    if isinstance(content, dict):
        return _extract_text_from_content([content])
    return None

def _is_real_user_turn(rec):

```

```

if not isinstance(rec, dict):
    return False
if rec.get("type") != "user":
    return False
if rec.get("isSidechain") or rec.get("isMeta"):
    return False
msg = rec.get("message") if isinstance(rec.get("message"), dict) else rec
content = msg.get("content") if isinstance(msg, dict) else None
if isinstance(content, list):
    non_tool = [c for c in content
                  if isinstance(c, dict)
                  and c.get("type") not in ("tool_result",)]
    if not non_tool:
        return False
return True

def read_last_assistant_turn(transcript_path):
    """Retries up to 5 times because the Stop hook can fire before
    the assistant record is flushed to the transcript file."""
    if not transcript_path or not os.path.exists(transcript_path):
        dbg(f"no transcript at {transcript_path}")
        return None

    attempts = 5
    delay = 0.3

    for attempt in range(attempts):
        try:
            with open(transcript_path, "r") as f:
                records = []
                for line in f:
                    line = line.strip()
                    if not line:
                        continue
                    try:
                        records.append(json.loads(line))
                    except Exception:
                        continue

            last_user_idx = None
            for i in range(len(records) - 1, -1, -1):
                if _is_real_user_turn(records[i]):
                    last_user_idx = i
                    break

            if last_user_idx is None:
                dbg(f"attempt {attempt+1}: no user prompt found")
                if attempt < attempts - 1:

```

```

        time.sleep(delay)
        continue
    return None

collected = []
for j in range(last_user_idx + 1, len(records)):
    r = records[j]
    if not isinstance(r, dict):
        continue
    t = r.get("type")
    if t == "user" and _is_real_user_turn(r):
        break
    if t != "assistant":
        continue
    msg = r.get("message") if isinstance(r.get("message"), dict) else r
    text = _extract_text_from_content(msg.get("content"))
    if text:
        collected.append(text)

if collected:
    if len(collected) > 1:
        tail = collected[-1].strip().lower()
        if tail.startswith("no response"):
            collected.pop()
    result = "\n\n".join(collected).strip()
    if result:
        dbg(f"found assistant text on attempt {attempt+1},
len={len(result)}")
        return result

    dbg(f"attempt {attempt+1}: no text yet, retrying")
    if attempt < attempts - 1:
        time.sleep(delay)

except Exception as e:
    dbg(f"attempt {attempt+1} failed: {e}")
    if attempt < attempts - 1:
        time.sleep(delay)

dbg("all retries exhausted, returning None")
return None

def extract(data):
    event = data.get("hook_event_name", "Unknown")
    transcript = data.get("transcript_path")
    row = {
        "session_id": data.get("session_id", "unknown"),
        "event_type": event, "agent": "main", "role": None, "content": None,

```

```

        "tool_name": None, "tool_input": None, "tool_output": None,
        "is_error": False, "error_message": None,
        "raw_payload": data, "transcript_path": transcript,
    }
    if event == "UserPromptSubmit":
        row["role"] = "user"
        row["content"] = data.get("prompt", "")
    elif event == "Stop":
        row["role"] = "assistant"
        row["content"] = read_last_assistant_turn(transcript)
    elif event == "SubagentStop":
        row["role"] = "assistant"
        row["agent"] = data.get("agent_name") or data.get("subagent_type") or
"subagent"
        row["content"] = read_last_assistant_turn(transcript)
    elif event == "PreToolUse":
        row["role"] = "tool"
        row["tool_name"] = data.get("tool_name")
        row["tool_input"] = data.get("tool_input")
    elif event == "PostToolUse":
        row["role"] = "tool"
        row["tool_name"] = data.get("tool_name")
        row["tool_input"] = data.get("tool_input")
        tr = data.get("tool_response") or {}
        row["tool_output"] = tr if isinstance(tr, (dict, list)) else {"raw": str(tr)}
        err = None
        if isinstance(tr, dict):
            if tr.get("is_error") or tr.get("error"):
                err = tr.get("error") or tr.get("message") or "tool reported error"
            elif isinstance(tr.get("stderr"), str) and tr["stderr"].strip():
                err = tr["stderr"][:2000]
        if err:
            row["is_error"] = True
            row["error_message"] = str(err)
    elif event == "Notification":
        row["role"] = "system"
        row["content"] = data.get("message", "")
    else:
        row["content"] = data.get("message") or data.get("prompt")
    return row

def jdump(v):
    return json.dumps(v) if v is not None else None

def main():
    try:
        raw = sys.stdin.read()

```

```

        dbg(f"stdin received: {len(raw)} bytes")
    except Exception as e:
        log_error(f"stdin read failed: {e}\n{traceback.format_exc()}")
        sys.exit(0)

    if not raw.strip():
        dbg("stdin was empty – exiting")
        sys.exit(0)

    try:
        data = json.loads(raw)
        dbg(f"parsed: event={data.get('hook_event_name')}")
    except Exception as e:
        log_error(f"json parse failed: {e}\nraw was: {raw[:500]}")
        sys.exit(0)

    try:
        row = extract(data)
        dbg(f"row: type={row['event_type']}, role={row['role']}, "
            f"content_len={len(row['content']) if row['content'] else 0}")
    except Exception as e:
        log_error(f"extract failed: {e}\n{traceback.format_exc()}")
        sys.exit(0)

    try:
        conn = psycopg2.connect(**DB_CONFIG)
        conn.autocommit = True
        cur = conn.cursor()
        cur.execute("""
            INSERT INTO cc_sessions (session_id, cwd, project_dir, last_seen_at)
            VALUES (%s, %s, %s, NOW())
            ON CONFLICT (session_id) DO UPDATE SET last_seen_at = NOW()
            """, (row["session_id"], data.get("cwd"),
                data.get("project_dir") or data.get("cwd")))
        cur.execute("""
            INSERT INTO cc_events (
                session_id, event_type, agent, role, content,
                tool_name, tool_input, tool_output,
                is_error, error_message, raw_payload, transcript_path
            ) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
            """, (row["session_id"], row["event_type"], row["agent"], row["role"],
                row["content"], row["tool_name"],
                Json(row["tool_input"]) if row["tool_input"] is not None else None,
                Json(row["tool_output"]) if row["tool_output"] is not None else None,
                row["is_error"], row["error_message"],
                Json(row["raw_payload"]) if row["raw_payload"] is not None else None,
                row["transcript_path"]))
        cur.close()
        conn.close()

```

```

        dbg(f"INSERT OK for {row['event_type']}")
    except Exception as e:
        log_error(f"db op failed: {e}\n{traceback.format_exc()}")
        sys.exit(0)

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        log_error(f"unhandled: {e}\n{traceback.format_exc()}")
        sys.exit(0)

```

## Option B — MySQL logger script

Save this as `~/claude/log-to-db.py`. Replace `YOUR_PASSWORD_HERE` with your actual password.

```

#!/usr/bin/env python3
"""Claude Code -> MySQL logger with debug + retry on transcript read."""
import sys, os, json, time, traceback, datetime

DB_CONFIG = {
    "user": "claude",
    "password": "YOUR_PASSWORD_HERE", # <-- put your real password here
    "host": "localhost",
    "port": 3306,
    "database": "claude_logs",
}

DEBUG_LOG = os.path.expanduser("~/claude/logger-debug.log")
ERR_LOG = os.path.expanduser("~/claude/logger-errors.log")

def dbg(msg):
    try:
        with open(DEBUG_LOG, "a") as f:
            f.write(f"[{datetime.datetime.utcnow().isoformat()}] {msg}\n")
    except Exception:
        pass

def log_error(msg):
    try:
        with open(ERR_LOG, "a") as f:
            f.write(f"[{datetime.datetime.utcnow().isoformat()}] {msg}\n")
    except Exception:
        pass

```

```

dbg(f"=== script started, pid={os.getpid()}, python={sys.executable} ===")

try:
    import mysql.connector
    dbg("mysql.connector imported OK")
except Exception as e:
    dbg(f"FAILED to import mysql.connector: {e}")
    log_error(f"import failed: {e}\n{traceback.format_exc()}")
    sys.exit(0)

def _extract_text_from_content(content):
    if content is None:
        return None
    if isinstance(content, str):
        return content.strip() or None
    if isinstance(content, list):
        parts = []
        for c in content:
            if not isinstance(c, dict):
                continue
            ctype = c.get("type")
            if ctype == "text" and c.get("text"):
                parts.append(c["text"])
            elif ctype in ("thinking", "tool_use", "tool_result"):
                continue
            elif "text" in c and isinstance(c["text"], str):
                parts.append(c["text"])
        joined = "\n".join(p for p in parts if p).strip()
        return joined or None
    if isinstance(content, dict):
        return _extract_text_from_content([content])
    return None

def _is_real_user_turn(rec):
    if not isinstance(rec, dict):
        return False
    if rec.get("type") != "user":
        return False
    if rec.get("isSidechain") or rec.get("isMeta"):
        return False
    msg = rec.get("message") if isinstance(rec.get("message"), dict) else rec
    content = msg.get("content") if isinstance(msg, dict) else None
    if isinstance(content, list):
        non_tool = [c for c in content
                     if isinstance(c, dict)
                     and c.get("type") not in ("tool_result",)]

```

```

        if not non_tool:
            return False
    return True

def read_last_assistant_turn(transcript_path):
    """Retries up to 5 times because the Stop hook can fire before
    the assistant record is flushed to the transcript file."""
    if not transcript_path or not os.path.exists(transcript_path):
        dbg(f"no transcript at {transcript_path}")
        return None

    attempts = 5
    delay = 0.3

    for attempt in range(attempts):
        try:
            with open(transcript_path, "r") as f:
                records = []
                for line in f:
                    line = line.strip()
                    if not line:
                        continue
                    try:
                        records.append(json.loads(line))
                    except Exception:
                        continue

            last_user_idx = None
            for i in range(len(records) - 1, -1, -1):
                if _is_real_user_turn(records[i]):
                    last_user_idx = i
                    break

            if last_user_idx is None:
                dbg(f"attempt {attempt+1}: no user prompt found")
                if attempt < attempts - 1:
                    time.sleep(delay)
                    continue
                return None

            collected = []
            for j in range(last_user_idx + 1, len(records)):
                r = records[j]
                if not isinstance(r, dict):
                    continue
                t = r.get("type")
                if t == "user" and _is_real_user_turn(r):
                    break

```

```

        if t != "assistant":
            continue
        msg = r.get("message") if isinstance(r.get("message"), dict) else r
        text = _extract_text_from_content(msg.get("content"))
        if text:
            collected.append(text)

    if collected:
        if len(collected) > 1:
            tail = collected[-1].strip().lower()
            if tail.startswith("no response"):
                collected.pop()
            result = "\n\n".join(collected).strip()
            if result:
                dbg(f"found assistant text on attempt {attempt+1},
len={len(result)}")
                return result

        dbg(f"attempt {attempt+1}: no text yet, retrying")
        if attempt < attempts - 1:
            time.sleep(delay)

    except Exception as e:
        dbg(f"attempt {attempt+1} failed: {e}")
        if attempt < attempts - 1:
            time.sleep(delay)

    dbg("all retries exhausted, returning None")
    return None

def extract(data):
    event = data.get("hook_event_name", "Unknown")
    transcript = data.get("transcript_path")
    row = {
        "session_id": data.get("session_id", "unknown"),
        "event_type": event, "agent": "main", "role": None, "content": None,
        "tool_name": None, "tool_input": None, "tool_output": None,
        "is_error": False, "error_message": None,
        "raw_payload": data, "transcript_path": transcript,
    }
    if event == "UserPromptSubmit":
        row["role"] = "user"
        row["content"] = data.get("prompt", "")
    elif event == "Stop":
        row["role"] = "assistant"
        row["content"] = read_last_assistant_turn(transcript)
    elif event == "SubagentStop":
        row["role"] = "assistant"

```

```

        row["agent"] = data.get("agent_name") or data.get("subagent_type") or
"subagent"
        row["content"] = read_last_assistant_turn(transcript)
    elif event == "PreToolUse":
        row["role"] = "tool"
        row["tool_name"] = data.get("tool_name")
        row["tool_input"] = data.get("tool_input")
    elif event == "PostToolUse":
        row["role"] = "tool"
        row["tool_name"] = data.get("tool_name")
        row["tool_input"] = data.get("tool_input")
        tr = data.get("tool_response") or {}
        row["tool_output"] = tr if isinstance(tr, (dict, list)) else {"raw": str(tr)}
        err = None
        if isinstance(tr, dict):
            if tr.get("is_error") or tr.get("error"):
                err = tr.get("error") or tr.get("message") or "tool reported error"
            elif isinstance(tr.get("stderr"), str) and tr["stderr"].strip():
                err = tr["stderr"][:2000]
        if err:
            row["is_error"] = True
            row["error_message"] = str(err)
    elif event == "Notification":
        row["role"] = "system"
        row["content"] = data.get("message", "")
    else:
        row["content"] = data.get("message") or data.get("prompt")
    return row

def jdump(v):
    return json.dumps(v) if v is not None else None

def main():
    try:
        raw = sys.stdin.read()
        dbg(f"stdin received: {len(raw)} bytes")
    except Exception as e:
        log_error(f"stdin read failed: {e}\n{traceback.format_exc()}")
        sys.exit(0)

    if not raw.strip():
        dbg("stdin was empty – exiting")
        sys.exit(0)

    try:
        data = json.loads(raw)
        dbg(f"parsed: event={data.get('hook_event_name')}")

```

```

except Exception as e:
    log_error(f"json parse failed: {e}\nraw was: {raw[:500]}")
    sys.exit(0)

try:
    row = extract(data)
    dbg(f"row: type={row['event_type']}, role={row['role']}, "
        f"content_len={len(row['content']) if row['content'] else 0}")
except Exception as e:
    log_error(f"extract failed: {e}\n{traceback.format_exc()}")
    sys.exit(0)

try:
    conn = mysql.connector.connect(**DB_CONFIG)
    cur = conn.cursor()
    cur.execute("""
        INSERT INTO cc_sessions (session_id, cwd, project_dir, last_seen_at)
        VALUES (%s, %s, %s, CURRENT_TIMESTAMP)
        ON DUPLICATE KEY UPDATE last_seen_at = CURRENT_TIMESTAMP
    """, (row["session_id"], data.get("cwd"),
        data.get("project_dir") or data.get("cwd")))
    cur.execute("""
        INSERT INTO cc_events (
            session_id, event_type, agent, role, content,
            tool_name, tool_input, tool_output,
            is_error, error_message, raw_payload, transcript_path
        ) VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
    """, (row["session_id"], row["event_type"], row["agent"], row["role"],
        row["content"], row["tool_name"],
        jdump(row["tool_input"]), jdump(row["tool_output"]),
        row["is_error"], row["error_message"],
        jdump(row["raw_payload"]), row["transcript_path"]))
    conn.commit()
    cur.close()
    conn.close()
    dbg(f"INSERT OK for {row['event_type']}")
except Exception as e:
    log_error(f"db op failed: {e}\n{traceback.format_exc()}")
    sys.exit(0)

if __name__ == "__main__":
    try:
        main()
    except Exception as e:
        log_error(f"unhandled: {e}\n{traceback.format_exc()}")
        sys.exit(0)

```

## Step 3 — Set permissions

The script must be executable AND readable only by you (the password is inside it):

```
chmod 700 ~/.claude/log-to-db.py
```

**Important:** Use chmod 700, not chmod 600. The 7 includes execute permission, which is needed for the hook to run the script. If you accidentally use 600 you'll see 'permission denied' errors and nothing will log.

## Step 4 — Register the hooks

Create ~/.claude/settings.json with the following contents. This is the global config — every Claude Code session on this machine will use it.

```
{
  "hooks": {
    "UserPromptSubmit": [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
    "Stop":               [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
    "SubagentStop":       [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
    "PreToolUse":         [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
    "PostToolUse":        [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
    "Notification":       [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
    "SessionStart":       [{ "hooks": [{ "type": "command", "command": "~/.claude/log-to-db.py" }] }],
  }
}
```

## Step 5 — Quick standalone test

Before opening Claude Code, verify the script can connect to the database on its own:

```
echo '{"hook_event_name":"Test","session_id":"manual-test"}' | ~/.claude/log-to-db.py
cat ~/.claude/logger-debug.log
```

You should see lines ending with 'INSERT OK for Test'. If yes, clean up the test row:

### PostgreSQL:

```
psql -U claude -d claude_logs -h localhost -c "
DELETE FROM cc_events WHERE session_id = 'manual-test';
DELETE FROM cc_sessions WHERE session_id = 'manual-test';"
```

### MySQL:

```
mysql -u claude -p claude_logs -e "
DELETE FROM cc_events WHERE session_id = 'manual-test';"
```

```
DELETE FROM cc_sessions WHERE session_id = 'manual-test';"
```

## Step 6 — Test in Claude Code

Fully restart Claude Code so it loads the new hooks:

- VS Code extension: Cmd+Q (Mac) / fully quit, then reopen
- CLI: exit the session, run claude again

Then send any message such as “hello”. After Claude responds, check the database.

### PostgreSQL:

```
psql -U claude -d claude_logs -h localhost -c "  
SELECT id, event_type, role, LEFT(content, 100) AS preview  
FROM cc_events ORDER BY id DESC LIMIT 10;"
```

### MySQL:

```
mysql -u claude -p claude_logs -e "  
SELECT id, event_type, role, LEFT(content, 100) AS preview  
FROM cc_events ORDER BY id DESC LIMIT 10;"
```

You should see a sequence like SessionStart → UserPromptSubmit → Stop, with the Stop row containing Claude's actual reply text in the preview column.

## Using the logs

Useful queries once data is flowing in. These work the same on PostgreSQL and MySQL — only the JSON syntax and date math differ slightly.

### Replay a single session in order

```
SELECT timestamp, role, agent, event_type, content, tool_name
FROM cc_events
WHERE session_id = 'abc-123'
ORDER BY id;
```

### All errors from the last 24 hours

#### PostgreSQL:

```
SELECT timestamp, session_id, tool_name, error_message
FROM cc_events
WHERE is_error AND timestamp > NOW() - INTERVAL '1 day'
ORDER BY timestamp DESC;
```

#### MySQL:

```
SELECT timestamp, session_id, tool_name, error_message
FROM cc_events
WHERE is_error = TRUE AND timestamp > NOW() - INTERVAL 1 DAY
ORDER BY timestamp DESC;
```

### Tool usage breakdown per agent

```
SELECT agent, tool_name, COUNT(*) AS uses
FROM cc_events
WHERE tool_name IS NOT NULL
GROUP BY agent, tool_name
ORDER BY uses DESC;
```

### All sessions for a specific project

```
SELECT s.session_id, s.started_at, COUNT(e.id) AS events
FROM cc_sessions s
LEFT JOIN cc_events e ON s.session_id = e.session_id
WHERE s.project_dir = '/Users/you/projects/my-app'
GROUP BY s.session_id, s.started_at
ORDER BY s.started_at DESC;
```

# Troubleshooting

This section covers every issue we hit while building and testing this setup. If something isn't working, scan through here first — the answer is almost certainly below.

## Diagnostic logs

The script writes two log files that tell you exactly what's happening:

- `~/.claude/logger-debug.log` — every step the script takes, success or failure. Always check here first.
- `~/.claude/logger-errors.log` — full Python tracebacks when something crashes.

View them with:

```
tail -30 ~/.claude/logger-debug.log
cat ~/.claude/logger-errors.log
```

## Issue: nothing logs at all (no debug log either)

Symptoms: no rows in the database, `~/.claude/logger-debug.log` doesn't exist or is empty, `~/.claude/logger-errors.log` is also empty.

This means the script never ran. Check in this order:

### 1. Permissions — most common cause

If you accidentally ran `chmod 600` (instead of `700`), the script lost execute permission. Fix:

```
chmod 700 ~/.claude/log-to-db.py
ls -la ~/.claude/log-to-db.py
```

You should see `-rwx-----` at the start. If there's no `x` in the first triplet, the script can't run.

### 2. Manual sanity check

```
echo '{}' | ~/.claude/log-to-db.py
cat ~/.claude/logger-debug.log
```

If this prints 'permission denied', go back to step 1. If it prints a Python error, the script has a syntax issue — re-paste it from this guide.

### 3. settings.json validity

```
python3 -m json.tool ~/.claude/settings.json
```

If this throws an error, your hooks file is malformed and Claude Code is silently ignoring it. Re-paste it from Step 4.

### 4. Restart Claude Code fully

Settings.json is loaded once at startup. Edits while Claude Code is running don't take effect. `Cmd+Q` the app or fully exit the CLI, then reopen.

## Issue: 'Access denied' errors in the error log

The script ran but couldn't authenticate. The traceback in `~/.claude/logger-errors.log` will say something like:

```
Access denied for user 'claude'@'localhost' (using password: YES)
```

### Common causes

- **Special characters in the password** — characters like `#`, `;`, `=` can be misinterpreted depending on how the password is loaded. With our script (password directly inline as a Python string) this is normally fine, but if you copied the password into nano or another editor that reformatted it, characters may have been altered. Verify by typing the password manually in `mysql -u claude -p`.
- **Wrong host or auth plugin** — check the user's host with:

```
mysql -u root -p -e "
SELECT user, host, plugin FROM mysql.user WHERE user='claude';"
```

It should show `'claude | localhost | caching_sha2_password'` (or `mysql_native_password`). If you see `'auth_socket'`, fix it with:

```
ALTER USER 'claude'@'localhost'
  IDENTIFIED WITH caching_sha2_password BY 'your_password';
FLUSH PRIVILEGES;
```

## Issue: Stop event saves NULL or 'No response requested.' for content

The script writes the row, but the assistant's reply is empty or shows a synthetic placeholder. This is the race condition the retry logic handles — Claude Code's Stop hook fires the moment streaming finishes, but the transcript file write happens slightly after.

If you see this, you may have an older version of the script without retries. Re-paste the version from this guide — it includes 5 retries with 300ms delays. Then check the debug log:

```
grep -E "attempt|found assistant" ~/.claude/logger-debug.log | tail -10
```

You should see lines like `'found assistant text on attempt 1, len=327'`. If it says `'attempt 2'` or `'attempt 3'`, the retry logic is doing its job.

## Issue: session\_id and event\_type say 'unknown'

The script ran but received no real data on stdin. This is harmless — it usually happens when you run the manual test command (`echo '{}' | ...`) which sends an empty JSON object. Real hook events always include `session_id` and `hook_event_name`.

Clean up these test rows:

```
-- PostgreSQL
DELETE FROM cc_events WHERE event_type IN ('Unknown', 'Test');
DELETE FROM cc_sessions WHERE session_id IN ('unknown', 'manual-test');

-- MySQL syntax is identical
```

## Issue: 'mysql.connector' or 'psycopg2' module not found

The driver is installed for a different Python interpreter than the one running your script. Check which Python the script uses:

```
head -1 ~/.claude/log-to-db.py
which python3
```

Then verify the driver is installed for that Python:

```
/usr/bin/python3 -c "import mysql.connector; print('OK')"
```

# or

```
/usr/bin/python3 -c "import psycopg2; print('OK')"
```

If it errors with `ModuleNotFoundError`, install the driver for that exact Python:

```
/usr/bin/python3 -m pip install mysql-connector-python
```

# or

```
/usr/bin/python3 -m pip install psycopg2-binary
```

## Sanity-check that hooks fire at all

If you suspect Claude Code isn't even calling the hook, replace the script command temporarily with a debug echo:

```
# Backup the real settings
cp ~/.claude/settings.json ~/.claude/settings.json.bak

# Replace temporarily
cat > ~/.claude/settings.json <<'EOF'
{
  "hooks": {
    "UserPromptSubmit": [{ "hooks": [{ "type": "command", "command": "tee -a
~/.claude/hook-debug.log" }] }],
    "Stop": [{ "hooks": [{ "type": "command", "command": "tee -a
~/.claude/hook-debug.log" }] }]}
}
EOF

> ~/.claude/hook-debug.log
```

Restart Claude Code, send a message, then:

```
cat ~/.claude/hook-debug.log
```

If raw JSON appears in this file, hooks are firing — the issue is in the Python script. If the file stays empty, hooks aren't firing at all (re-check settings.json syntax and that you fully restarted Claude Code).

Restore the real settings when done:

```
mv ~/.claude/settings.json.bak ~/.claude/settings.json
```

## Security & privacy notes

- Conversations may contain secrets, API keys, or proprietary code — treat the log database as sensitive data.
- The database password is in plaintext inside `~/claude/log-to-db.py`. The `chmod 700` ensures only your user account can read it, but don't commit this file to git or share screenshots of it.
- Never commit `~/claude/settings.json` to git either.
- Consider enabling encryption-at-rest on the database, especially on shared machines.
- If you need to disable logging temporarily, rename `~/claude/settings.json` to `settings.json.off`, restart Claude Code, and no events will be captured until you rename it back.
- For team/shared logging, point the script at a remote DB over SSL — but be aware that team members' prompts will be visible to anyone with DB access.
- Old logs accumulate over time. Consider a periodic cleanup job: `DELETE FROM cc_events WHERE timestamp < NOW() - INTERVAL '90 days';` (PostgreSQL) or with `INTERVAL 90 DAY` (MySQL).

## Summary

Once set up, this captures every Claude Code interaction across both the CLI and the VS Code extension, on every project, automatically. The configuration is global — set it up once per machine and forget about it.

The setup has been tested end-to-end and handles the tricky parts: stdin parsing edge cases, the transcript-write race condition on Stop events, content extraction across nested message shapes, and synthetic placeholder filtering. The debug log will tell you exactly what's happening at every step if anything goes wrong.

To verify it's truly global, run Claude Code from two different project folders and check that both sessions appear in `cc_sessions` with different `cwd` values, all flowing into the same database.